

FINAL TECHNICAL REPORT

IT3051 – FUNDAMENTALS OF DATA MINING

MINI PROJECT 2026



Online Shoppers Purchase-Intent Prediction System

Student Name	Student ID
H.N. Hapuarachchi	IT23713062
Dewmin W. A. V	IT23733930
Vihigum M. H. D. C	IT23699380
Vidyanjani H. M. R. M	IT23690202

Executive Summary

This work describes an end-to-end binary classification model to predict the intention to make a purchase in an online shopping session. The project includes defining the task, selecting a dataset, exploratory data analysis (EDA), preprocessing, feature engineering, multiple models building, tuning and selection, backend development, frontend development, testing, and interpretation.

The project uses the UCI Online Shoppers Purchasing Intention Dataset with 12,330 sessions and 17 predictors and one target – the Revenue. There are 1,908 purchase sessions (15.5%). During EDA, 92 duplicates and missing values in such features as BounceRates (7%), Administrative_Duration (6%), ProductRelated_Duration (5%), and VisitorType (4%) were found.

Seven different algorithm families have been explored. The random forest algorithm showed the best baseline PR-AUC value (0.746) and also showed the highest tuned cross-validation PR-AUC (0.753). On the unseen test set the ROC-AUC and PR-AUC reached 0.938 and 0.772 correspondingly. The optimized threshold equals to 0.381 and the accuracy, precision, recall and F1 score were 0.897, 0.656, 0.720, and 0.687 respectively.

The final project is built on the FastAPI backend and the Streamlit frontend. The application verifies the input, preprocesses the data, and calculates the purchase probability and intent level. Additionally, the frontend suggests what actions the business should take depending on the predicted intent level. PageValues variable is the best predictor, therefore, the project is more suitable to be implemented as the late session predictor.

To evaluate robustness of the algorithm, a special component was developed to create a synthetic version of the dataset with Missing Completely At Random (MCAR) values for the four features. It gave the group an opportunity to try out different imputation strategies and evaluate how improving the value reconstruction process affects the classification. In summary, this project consists of the data quality analysis, leakages prevention, class imbalance analysis, optimization, explainability, and deployment processes in one reproducible pipeline.

Table of Contents

Executive Summary	2
1. Introduction and Problem Definition	4
2. Scenario and Stakeholder Requirements	5
3. Dataset Identification, Source and Validation	5
4. Data Understanding and Exploratory Data Analysis	6
5. Data Cleaning and Preprocessing	10
6. Feature Engineering and Feature Selection	12
7. Model Development and Algorithm Rationale.....	13
8. Model Evaluation and Comparison	13
9. Hyperparameter Tuning and Optimization	14
10. Final Model Selection and Justification	15
11. System Architecture and Implementation	18
12. Backend and Frontend Development.....	20
13. System Testing and Results	21
14. Limitations and Future Improvements.....	22
15. Group and Individual Contributions	23
16. Conclusion	24
References.....	25

1. Introduction and Problem Definition

1.1 Background

Web analytics systems collect information on page views, time spent on site, bounce and exit rate, types of visitors and sources of traffic. This data could be used to predict how probable it is for the user to convert into client.

1.2 Problem Definition

The objective is to forecast if a given session will result in a purchase or not. It is a supervised learning binary classification problem. If Revenue=True/1, it represents a purchasing session, whereas Revenue=False/0 represents a non-purchasing session.

1.3 Objectives

- Investigate data structure, quality, distributions and relationships.
- Build a reproducible preprocessing and feature-engineering pipeline.
- Compare multiple algorithms with metrics suitable for class imbalance.
- Tune suitable models and justify a final model using experimental evidence.
- Integrate the selected model into a functional backend and user-friendly frontend.

1.4 Scope

As the final model includes PageValues, due to its high prediction rate in experiments. Thus, the implemented version can be considered as a late-session or complete-session scoring. It would be needed to implement another model without PageValues for an early session intervention.

1.5 Technical Questions and Success Criteria

The project was guided by four technical questions:

- (1) which preprocessing strategy handles the deliberately introduced missing data without sacrificing predictive performance
- (2) which model family best separates purchasing from non-purchasing sessions under class imbalance
- (3) whether feature engineering, feature selection and imbalance treatment provide measurable improvement
- (4) whether the selected model can be deployed without changing the preprocessing logic used during training

Success criterion	How it was evaluated
Reliable discrimination	Primary ranking by cross-validated PR-AUC, supported by ROC-AUC, F1, precision and recall.
Generalization	Final evaluation on an untouched stratified 20% hold-out set.
Leakage control	All learned preprocessing fitted inside training folds; PageValues evaluated through explicit ablation.
Operational consistency	Serialized preprocessing + model pipeline reused unchanged by the FastAPI service.
Usable decision output	Probability converted to an optimized decision threshold and business-oriented intent/recommendation.

2. Scenario and Stakeholder Requirements

2.1 Scenario

An online store is looking to determine the probability of a sale based on the visitor's actions in order to decide the correct action: eliminate friction for highly-intent visitors, offer social proof or delivery assurance for potential buyers, suggest products for undecided visitors, or employ cheap retargeting for low intent visitors.

2.2 Stakeholders and Decisions

Stakeholder	Decision supported
Marketing team	Prioritize offers, retargeting and conversion campaigns.
E-commerce / UX team	Identify sessions that may benefit from checkout reassurance or reduced friction.
Business analysts	Understand behavioural variables associated with purchase outcomes.
System users	Obtain a clear probability, intent band and recommended action.

2.3 Requirements

- Accept the available 17 session fields and allow unknown optional fields.
- Validate numeric ranges and categorical values.
- Require at least five supplied fields before prediction.
- Apply the same preprocessing used during model training.
- Return probability, binary decision, threshold, intent level and recommended action.
- Warn users when many values are imputed.

3. Dataset Identification, Source and Validation

3.1 Dataset and Citation

Dataset: Online Shoppers Purchasing Intention Dataset, UCI Machine Learning Repository. The project README records: Sakar, C. & Kastro, Y. (2018), DOI 10.24432/C5F88Q, CC BY 4.0. Source:

<https://archive.ics.uci.edu/dataset/468/online+shoppers+purchasing+intention+dataset>

Characteristic	Value
Records	12,330
Predictors	17
Target	Revenue
Task	Binary classification
Non-purchase	10,422 (84.5%)
Purchase	1,908 (15.5%)

3.2 Suitability and Validation

The dataset fits well with the scenario, as each record is a customer session, while the Revenue variable provides a distinct purchase result. The dataset is large enough to be used for EDA, preprocessing, feature engineering, analysis of class imbalance, and model comparison/optimization. The project GitHub repository reveals further steps on the basis of this particular dataset.

3.3 Ethical and Privacy Considerations

The project data contains session attributes rather than direct personal identifiers in the supplied files. Predictions are probabilistic and should not be treated as certain statements about individuals. Any operational use should avoid manipulative or discriminatory targeting and should communicate uncertainty.

3.4 Project Dataset Version and Data Provenance

UCI dataset is a complete benchmark dataset. In the current mini project, the approved working version has been changed purposely to have MCAR missing values with a predetermined random seed. This makes sure that the analysis done on the missing values is treated as an experiment condition instead of a feature that existed in the UCI data set originally.

The working data set has 12,330 observations and 18 columns with missing values in Administrative Duration, Product Related Duration, Bounce Rates, and Visitor Type only. This has made sure that all results from the project are internally comparable as the approved modified data set is used in EDA, preprocessing, modelling and deployment.

4. Data Understanding and Exploratory Data Analysis

4.1 Structure and Missing Values

Field	Type	Unique	Missing
Administrative	float64	27	0
Administrative_Duration	float64	3175	740
Informational	float64	17	0
Informational_Duration	float64	1258	0
ProductRelated	float64	311	0
ProductRelated_Duration	float64	9121	616
BounceRates	float64	1783	863
ExitRates	float64	4777	0
PageValues	float64	2704	0
SpecialDay	float64	6	0

Month	object	10	0
OperatingSystems	object	8	0
Browser	object	13	0
Region	object	9	0
TrafficType	object	20	0
VisitorType	object	3	493
Weekend	object	2	0
Revenue	int64	2	0

The missing data are found in 4 variables: BounceRates 863 (7%), Administrative_Duration 740 (6%), ProductRelated_Duration 616 (5%), and VisitorType 493 (4%). The missing data pattern analysis indicated that the missingness pattern is consistent with the MCAR assumption.



Figure 1. Missing-value counts by feature

4.2 Duplicates and Class Balance

EDA found 92 duplicate rows (about 0.75%), which were removed before splitting to reduce cross-partition duplication. Revenue is imbalanced: only 15.5% of sessions are purchases. Therefore accuracy alone is misleading and PR-AUC, F1, recall, precision, balanced accuracy and ROC-AUC were used.

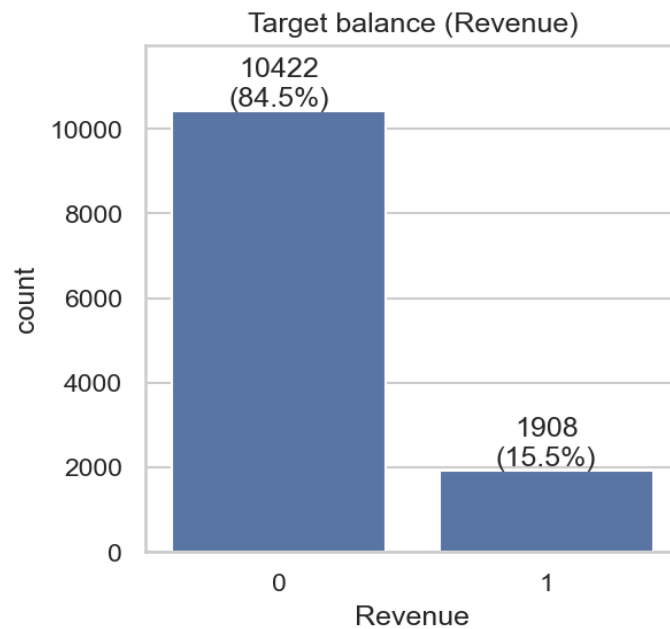


Figure 2. Target class balance

4.3 Distributions and Outliers

There are also some variables that display strong right-skewness and zero-inflation. The values for Administrative_Duration have skewness 5.61; Informational_Duration 7.58; ProductRelated_Duration 7.40; and PageValues 6.38. It is not likely that the outliers represent any obvious mistake, and therefore such an aggressive treatment would not be appropriate.

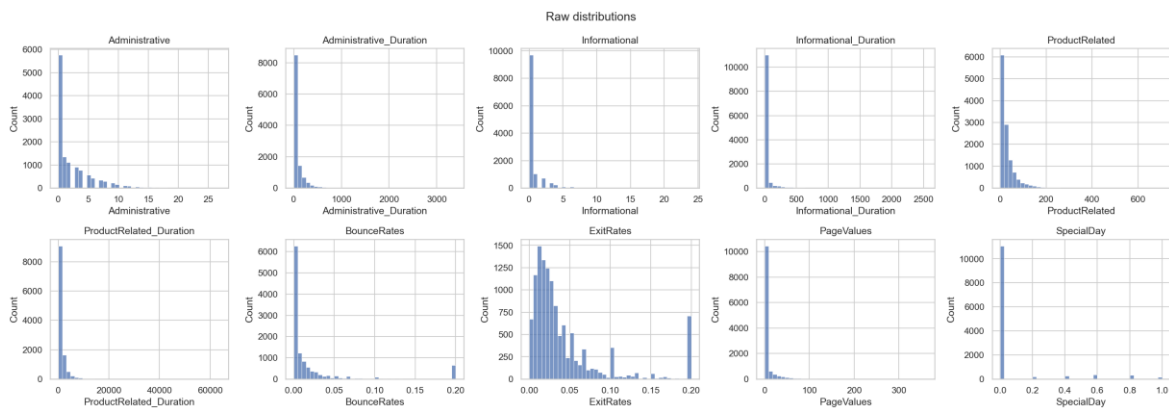


Figure 3. Raw numerical distributions

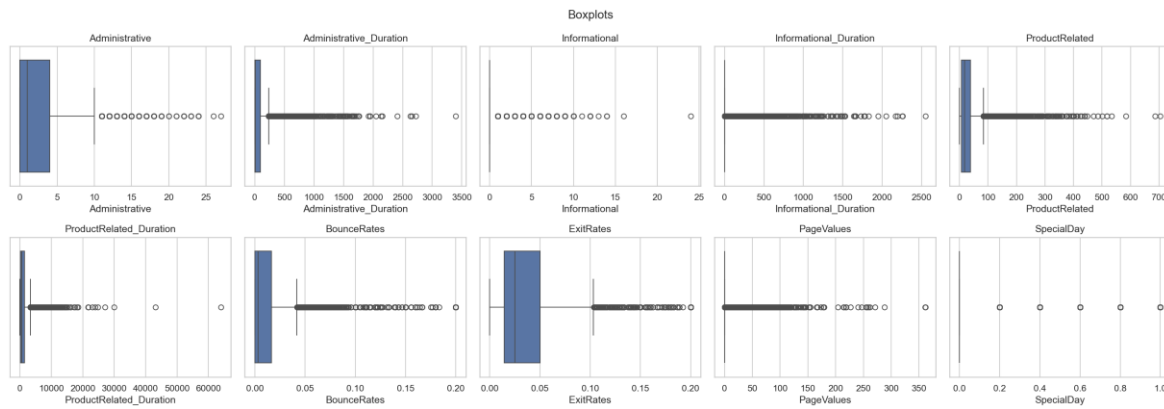


Figure 4. Numerical boxplots and extreme observations

4.4 Relationships with Revenue

PageValues shows the strongest relationship with Revenue (correlation 0.493; mutual information 0.166). ExitRates is negatively associated with purchase (-0.207), while ProductRelated and ProductRelated_Duration are positively associated. These findings later agree with model-based feature importance.

Feature	Correlation
PageValues	0.493
ExitRates	-0.207
ProductRelated	0.159
ProductRelated_Duration	0.152
BounceRates	-0.150
Administrative	0.139
Informational	0.095
Administrative_Duration	0.091

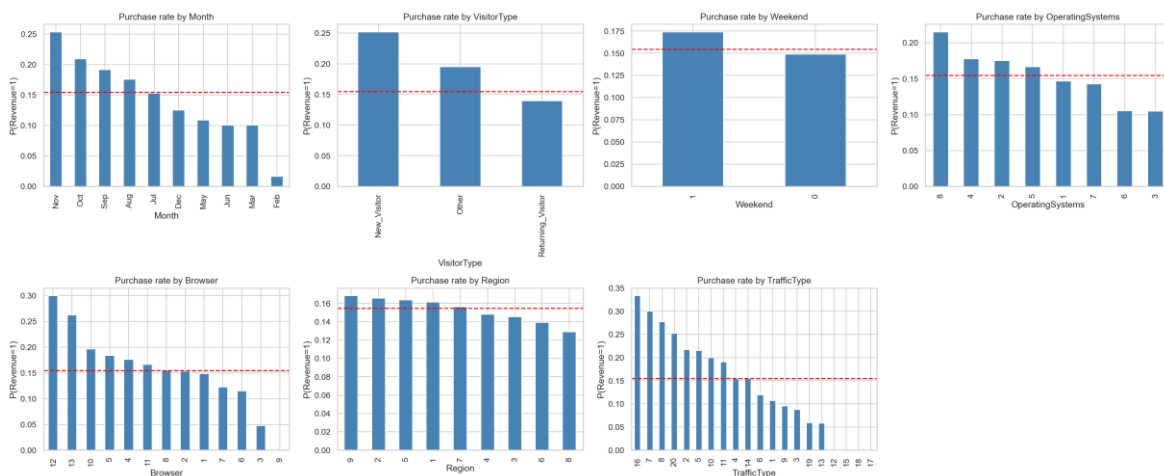


Figure 5. Purchase rate by categorical variables

4.5 Leakage Investigation

PageValues was considered to be explicitly testable because of its strong relationship with conversion rate. Removal of PageValues led to the deterioration of HistGradientBoosting PR-AUC from 0.738 to 0.349 and that of Random Forest PR-AUC from 0.742 to 0.368. However, the PageValue metric was kept because of the final nature of the model.

4.6 EDA Implications for Modelling

The results from the exploratory analysis phase played an important role in informing subsequent modelling approaches, instead of serving solely as a descriptive tool. Class imbalance led to PR-AUC being the main measure of performance, along with experimenting with class balancing and SMOTE. Right skewness led to trying log1p transformation. The presence of missing values led to running a comparison of imputation methods. The correlation between PageValues led to doing a scope ablation test for leakage.

5. Data Cleaning and Preprocessing

5.1 Split and Leakage Prevention

After duplicate removal, a stratified 80/20 hold-out split produced 9,790 training rows and 2,448 test rows, each with about 15.6% buyers. Model selection used stratified 5-fold cross-validation on training data. Preprocessing was embedded in scikit-learn pipelines so imputation, transformations, encoding and scaling were learned only from training folds.

5.2 Missing-Value Handling

Mean, median, KNN, and iterative imputation for numeric variables were compared, together with mode and an explicit Unknown category for VisitorType. Although KNN and iterative imputers sometimes reconstructed the missing values more accurately, downstream modelling performance was nearly unchanged. Median imputation for numeric variables and mode imputation for categorical variables were therefore selected for simplicity and speed. Deleting incomplete rows was rejected because it would reduce the 9,790 training rows to 7,805 without a meaningful performance gain.

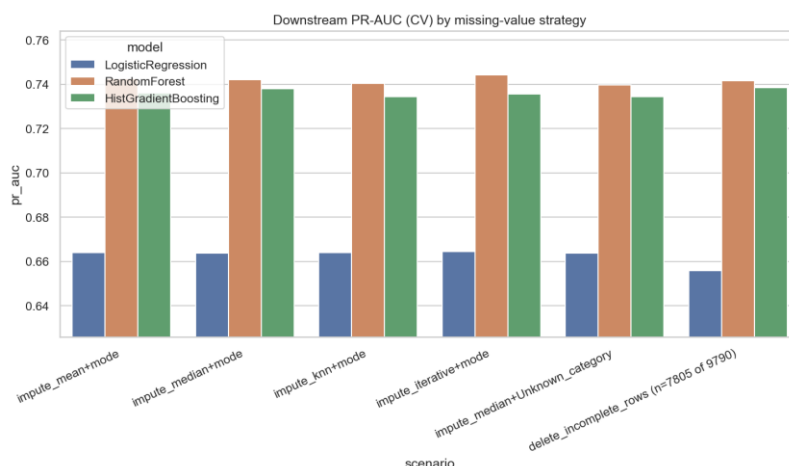


Figure 6. Downstream performance under missing-value strategies

5.3 Outliers, Encoding and Scaling

Positive numeric variables follow right skewness, and therefore apply the log1p transformation. Quantile capping and no transformation gave very close PR-AUCs to log1p transformation, hence the choice of log1p transformation to reflect right-skewed variables better. Categorical variables apply one hot encoding where rare levels are encoded to below 1%. The StandardScaler is applied to numeric variables, which is critical in logistic regression, SVM, and KNN algorithms.

5.4 Class Imbalance

The effects of class weighting and SMOTE were compared to no weighting. Without using any method to deal with the imbalance problem, the best baseline PR-AUC score obtained by Random Forest is 0.746. However, by using class weighting, some classifiers performed better with respect to recall, like HistGradientBoosting which got 0.780 recall, but had poor precision.

5.5 Final Preprocessing Decision Summary

Decision area	Alternatives evaluated	Final choice	Evidence / justification
Numeric missing values	Mean, median, KNN, iterative	Median	KNN/iterative improved reconstruction for some variables, but downstream CV performance was nearly unchanged; median was simpler and faster.
Categorical missing values	Mode, Unknown category	Mode	Stable downstream performance with lower complexity.
Incomplete-row deletion	Delete vs impute	Rejected	Would reduce training data from 9,790 to 7,805 rows with no meaningful performance gain.
Outliers	None, log1p, quantile cap	log1p	PR-AUC differences were negligible; log transform better represents highly skewed positive variables.
Encoding	One-hot with rare-level handling	One-hot	Categorical IDs are nominal rather than ordinal.
Scaling	No scaling vs StandardScaler where required	StandardScaler	Required by LR/SVM/KNN and harmless inside the shared pipeline.
Feature selection	None, mutual information, RF importance	None	Both selectors slightly underperformed retaining all prepared features.
Imbalance	None, class weights, SMOTE	None for final RF	Random Forest without resampling/weights had the best baseline PR-AUC.

6. Feature Engineering and Feature Selection

Engineered feature	Definition
TotalPages	Administrative + Informational + ProductRelated
TotalDuration	Sum of three duration fields
AvgTimePerPage	TotalDuration / TotalPages with zero-safe handling
ProductRelatedShare	ProductRelated / TotalPages
IsHolidaySeason	1 for Nov/Dec, else 0
HasPageValue	1 when PageValues > 0

Ablation experiments showed no measurable harm from the engineered features, so they were retained for interpretability. Mutual-information top-k and Random-Forest-importance selectors both slightly underperformed the no-selection pipeline; therefore no explicit feature-selection stage was used.

6.1 Feature-Engineering Rationale and Ablation Interpretation

These engineered variables describe the intensity of the behaviour in ways that are more easily understood by the stakeholders. These variables include TotalPages, TotalDuration, AvgTimePerPage which is the time normalized with respect to the browsing activity, ProductRelatedShare which represents the share of product-related activities, IsHolidaySeason, and HasPageValue which is a binary variable for the presence of page value information. From the results of the ablation study, it was seen that these variables do not affect the PR-AUC significantly and hence were retained for interpretability purposes.

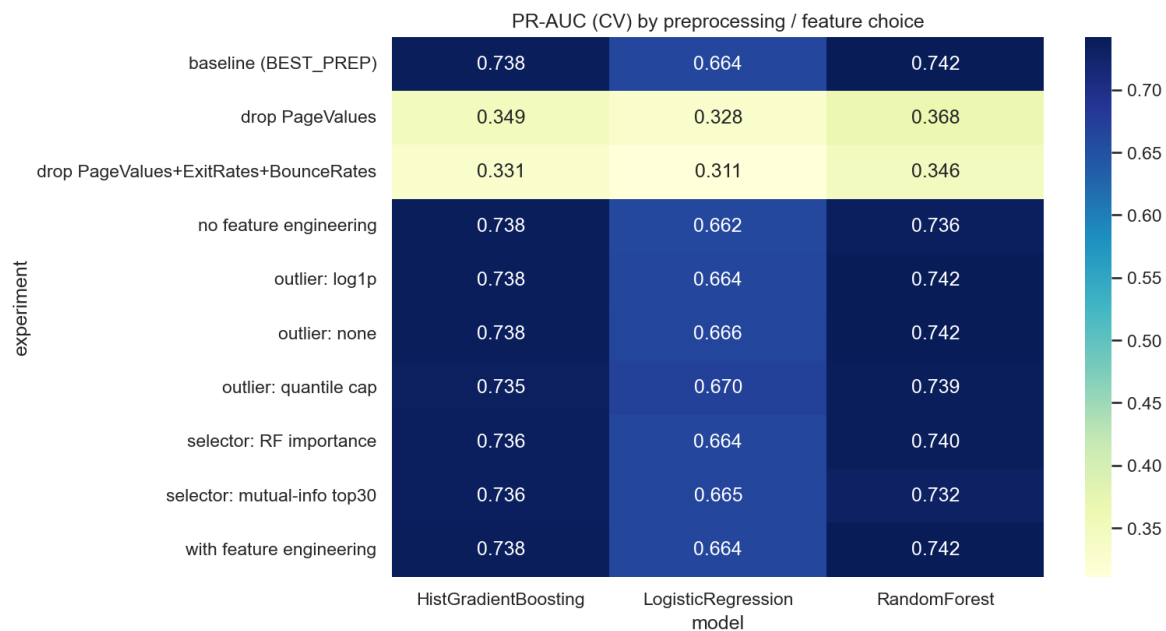


Figure 7. Feature and preprocessing ablation experiments

7. Model Development and Algorithm Rationale

Seven algorithm families were evaluated, exceeding the minimum requirement of four. The set spans linear, distance-based, single-tree, bagging, boosting and kernel approaches.

Algorithm	Rationale
Logistic Regression	Interpretable linear baseline for binary classification.
Decision Tree	Non-linear rules and interactions; useful single-tree benchmark.
Random Forest	Robust tree ensemble with non-linear modelling and feature importance.
HistGradientBoosting	Efficient boosting for complex non-linear patterns.
XGBoost	Regularized gradient boosting with strong tuning flexibility.
RBF-SVM	Non-linear margin-based classifier after scaling.
KNN	Distance-based non-parametric benchmark after scaling.

7.1 Metrics

PR-AUC was the primary ranking metric because purchases form only 15.5% of the data. ROC-AUC measures ranking across thresholds; F1 balances precision and recall; recall reflects how many buyers are detected; precision reflects how many positive interventions target actual buyers; balanced accuracy accounts for both classes.

7.2 Experimental Design

The comparison was set up to examine various modelling assumptions, and not just a variety of approaches based on the same algorithm. The Linear Baseline is provided by Logistic Regression; KNN examines whether points are locally similar; RBF-SVM includes a non-linear kernel boundary; Decision Tree presents an easy-to-understand model with high variance; Random Forest minimises variance via bagging; HistGradientBoosting and XGBoost capture complex non-linear interactions through boosting.

All comparisons of models used identical train/test split, preprocessing choices and stratified cross-validation methodology in order for the performance difference to be due to learning algorithm and imbalance strategy chosen.

8. Model Evaluation and Comparison

Model	Imbalance	F1	Recall	Precision	ROC-AUC	PR-AUC
RandomForest	none	0.654	0.591	0.733	0.928	0.746
RandomForest	class weight	0.641	0.566	0.741	0.929	0.742
HistGradientBoosting	none	0.642	0.582	0.717	0.928	0.739
HistGradientBoosting	class weight	0.669	0.780	0.586	0.927	0.738
HistGradientBoosting	smote	0.660	0.658	0.663	0.927	0.730
RandomForest	smote	0.678	0.717	0.643	0.926	0.721
SVM RBF	none	0.655	0.609	0.708	0.901	0.718
XGBoost	smote	0.652	0.631	0.675	0.922	0.715
XGBoost	none	0.632	0.579	0.696	0.920	0.712
XGBoost	class weight	0.656	0.691	0.625	0.919	0.711

Random Forest without imbalance adjustment achieved the best baseline PR-AUC (0.746).

HistGradientBoosting with class weighting provided the strongest recall among leading configurations but at lower precision. The single Decision Tree performed substantially worse than ensemble trees, supporting the use of bagging or boosting.

8.1 Interpretation of Baseline Results

The initial results show that the decision boundary is largely non-linear. Ensembles of trees were always superior to a lone Decision Tree, and were comparable on all three metrics. The better performance of the ensemble compared to the single tree implies that an ensemble of many trees reduces the variance and interaction of the following features: page value, exits, product-related actions, visitor type, and seasonality.

8.2 Precision-Recall Trade-off Under Imbalance Handling

The use of class balancing techniques and SMOTE resulted in improved recall but lower precision for many of the models. This is quite expected since a higher weighting of the minority class results in more positive predictions. The PR-AUC value was used for ranking the models instead of choosing the configuration with the highest recall to avoid biasing the results.

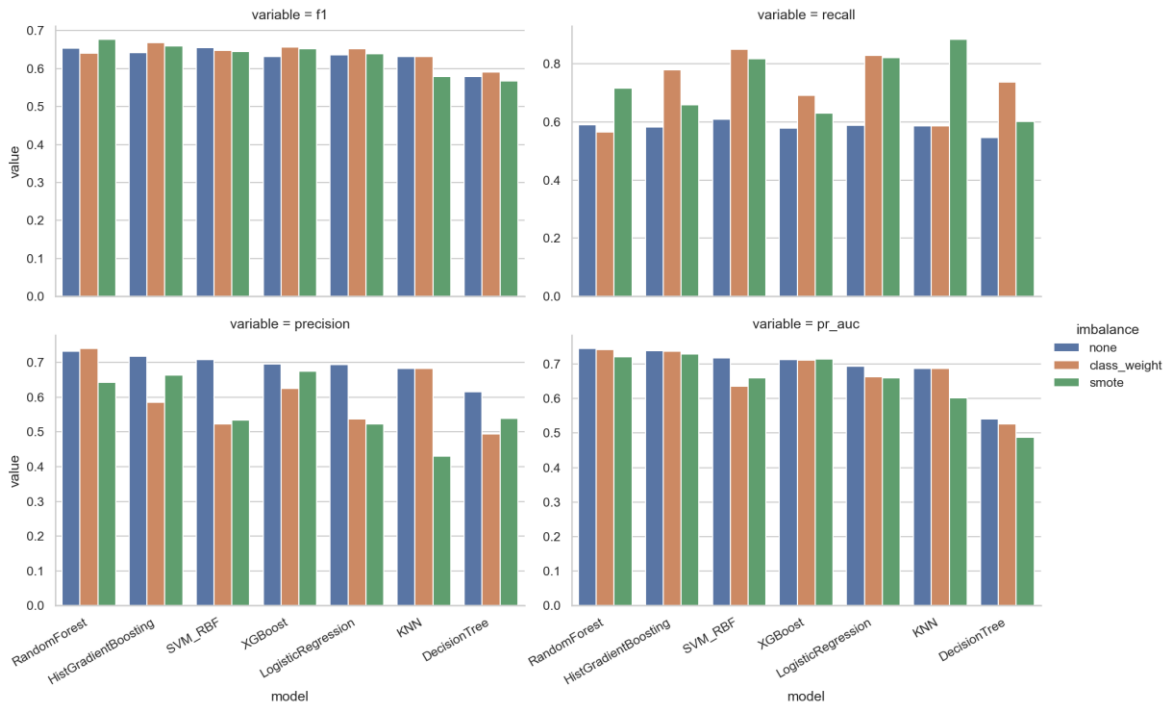


Figure 8. Baseline model comparison

9. Hyperparameter Tuning and Optimization

RandomizedSearchCV with 40 sampled candidates and stratified 5-fold cross-validation was used for the main tuning experiments, refitting on PR-AUC. This balances search coverage with computational cost.

Model	CV PR-AUC	CV ROC-AUC	CV F1	Best parameters (summary)
RandomForest	0.7526	0.9304	0.6467	{"class_weight": null, "max_depth": 12, "max_features": "sqrt", "min_samples_leaf": 5, "n_estimators": 100}
HistGradientBoosting	0.7514	0.9338	0.6514	{"class_weight": null, "l2_regularization": 0.014926318309209369, "learning_rate": 0.020777301988127}
XGBoost	0.7512	0.9327	0.6602	{"colsample_bytree": 0.847892199672541, "learning_rate": 0.03713861285134416, "max_depth": 12, "max_features": "sqrt", "min_samples_leaf": 5, "n_estimators": 100}

				"max_depth": 4, "min_c
SVM_RBF	0.7253	0.9073	0.6575	{"C": 28.34090429514775, "class_weight": null, "gamma": 0.00627773091695805}
KNN	0.7233	0.9208	0.6134	{"n_neighbors": 48, "p": 1, "weights": "distance"}
DecisionTree	0.7016	0.9123	0.6494	{"class_weight": null, "criterion": "gini", "max_depth": 6, "min_samples_leaf": 19, "min_samples_split": 2}
LogisticRegression	0.6668	0.9196	0.6560	{"C": 0.03334792728637585}

Random Forest achieved the highest tuned CV PR-AUC at 0.7526, followed closely by HistGradientBoosting at 0.7514 and XGBoost at 0.7512. XGBoost gained about 0.0366 PR-AUC from tuning, while tuned Logistic Regression was below its baseline PR-AUC.

9.1 Interpretation of Tuning Results

The tuned PR-AUC scores indicate that Random Forest, HistGradientBoosting and XGBoost are statistically tied within 0.002 PR-AUC approximately. Therefore, the project does not have a strong statistical advantage for the Random Forest method. Instead, final choice is made based on the combination of slight performance advantage and ease of implementation, low cost of training and interpretability. The XGBoost was more sensitive to the tuning than Logistic Regression; the latter was less affected by tuning, indicating a non-linear nature of the predictive structure.

RandomizedSearchCV searched through 40 random parameter configurations per tuned model class using stratified five folds with re-fit of the best performing configuration according to PR-AUC criterion. This corresponds to 200 model fits per tuned model family.

10. Final Model Selection and Justification

10.1 Selected Model

Random Forest was selected because it achieved the highest tuned CV PR-AUC (0.7526), remained strong across other metrics, and offers simpler global interpretation than the boosted alternatives. Selected parameters: n_estimators=424, max_depth=12, max_features='sqrt', min_samples_leaf=5, class_weight=None.

10.2 Threshold Optimization

The probability threshold was optimized using out-of-fold training predictions to maximize F1. The selected threshold was 0.380874 (0.381). Compared with 0.5, recall increased from 0.620 to 0.720, precision decreased from 0.762 to 0.656, and F1 increased slightly from 0.684 to 0.687.

Setting	Accuracy	Balanced Acc.	Precision	Recall	F1	ROC-AUC	PR-AUC
threshold=0.5	0.911	0.792	0.762	0.620	0.684	0.938	0.772
threshold=0.381	0.897	0.825	0.656	0.720	0.687	0.938	0.772

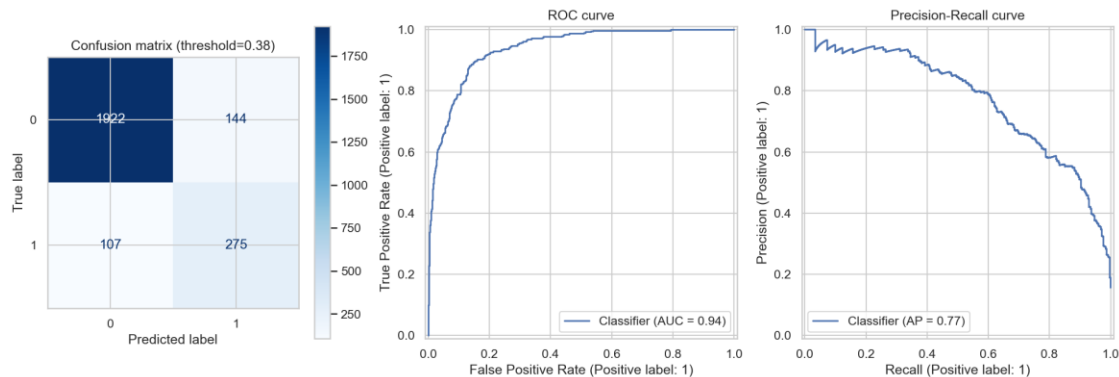


Figure 9. Final hold-out model evaluation

10.3 Interpretation

Permutation importance confirms PageValues as the dominant predictor (0.560), followed by Month (0.046), ExitRates (0.025), ProductRelated (0.025), Administrative (0.017), VisitorType (0.013), ProductRelated_Duration (0.012) and BounceRates (0.012).

10.4 Business Interpretation of the Decision Threshold

The optimized threshold of 0.381 is not merely a mathematical adjustment. Relative to the default 0.50 threshold, recall rises from 0.620 to 0.720 while precision falls from 0.762 to 0.656. In practical terms, the system catches more actual buyers but also flags more non-buyers as high intent. This trade-off is appropriate when the cost of a lightweight marketing action is lower than the opportunity cost of missing a likely purchaser. If interventions become expensive, the threshold should be re-optimized using explicit business costs rather than F1 alone.

10.5 Explainability and Model Scope

Permutation importance and SHAP both support the same central conclusion: PageValues dominates the model, while month, exit behaviour and product-related engagement provide secondary information. These explanations are associative rather than causal. They describe how the model uses observed variables and should not be interpreted as proof that changing a feature will directly cause a purchase.

Feature	Permutation importance
PageValues	0.5598
Month	0.0461
ExitRates	0.0254
ProductRelated	0.0248
Administrative	0.0169
VisitorType	0.0134
ProductRelated_Duration	0.0118
BounceRates	0.0116
Weekend	0.0032
Informational	0.0021

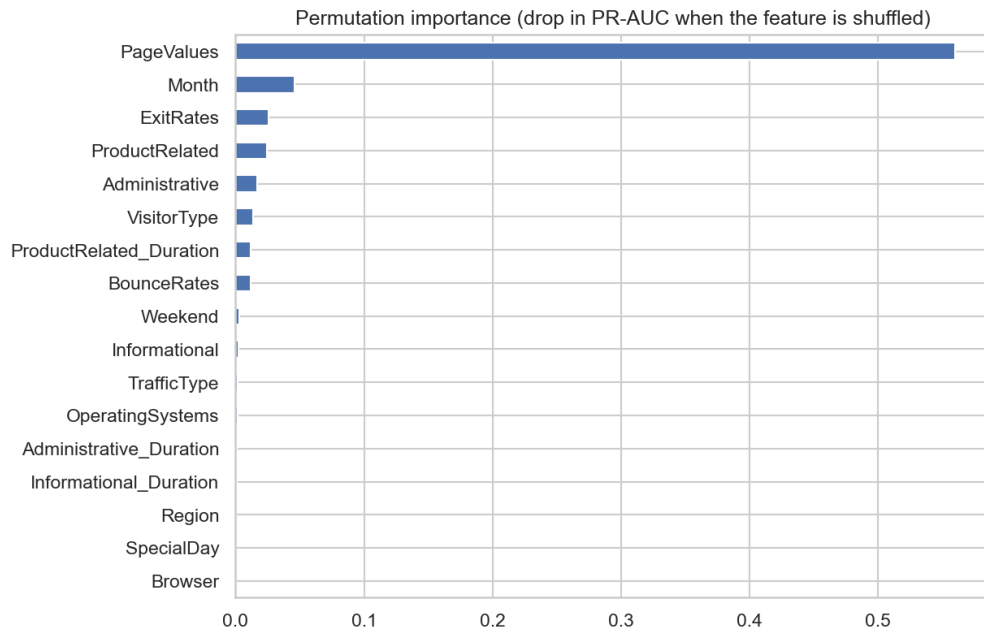


Figure 10. Final model permutation importance

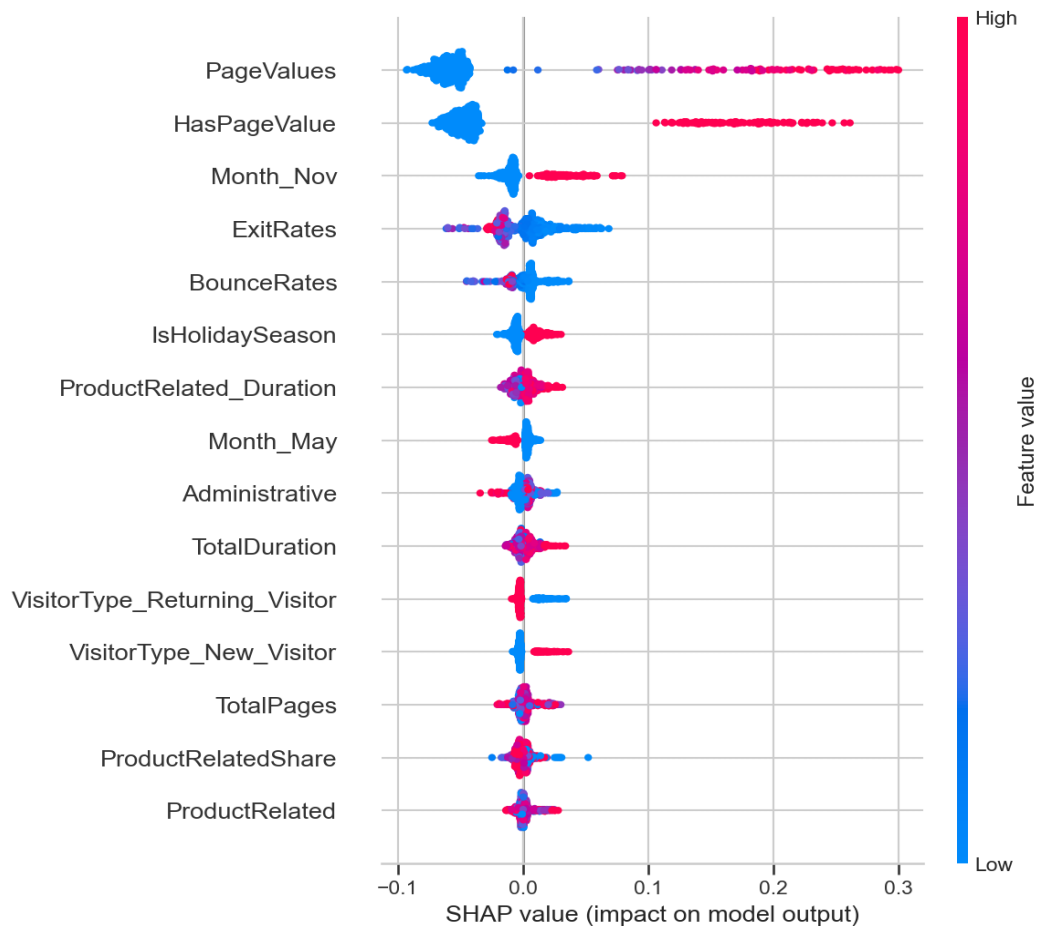


Figure 11. SHAP summary of final model

11. System Architecture and Implementation

The application uses a three-layer design: Streamlit frontend, FastAPI service layer, and a serialized scikit-learn preprocessing/model artefact. Keeping preprocessing inside the saved pipeline prevents training-serving mismatch.

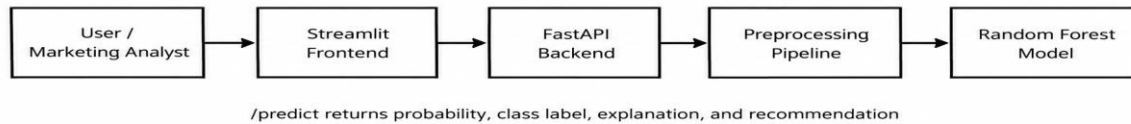


Figure 12. End-to-end system architecture and prediction flow

11.1 Repository and Component Structure

The repository separates exploratory work, reusable machine-learning logic and deployment code. The notebooks/ directory contains the three analysis notebooks; src/ contains preprocessing, training and evaluation functions; backend/ contains the FastAPI service and prediction logic; frontend/ contains the Streamlit application; models/ stores serialized model artefacts; data/ separates raw and optional processed data; and reports/ stores figures, tables and decision evidence. This structure reduces duplication and makes the final workflow easier to reproduce and explain during the viva.

Component	Primary role	
notebooks/01_eda.ipynb	Dataset understanding, quality checks and exploratory evidence.	
notebooks/02_preprocessing.ipynb	Imputation, outlier, feature, selector and leakage experiments.	
notebooks/03_modelling.ipynb	Baselines, tuning, threshold selection, interpretation and final model export.	
src/preprocessing.py	Shared data loading, schema enforcement and preprocessing pipeline.	
src/train.py / src/evaluate.py	Model definitions, tuning, metrics and evaluation helpers.	
backend/api.py / service.py	Request validation, model loading and prediction response.	
frontend/streamlit_app.py	User-facing data entry and business-friendly output.	
Layer	Technology	Responsibility
Frontend	Streamlit	Collect inputs and display probability, intent level, warnings and actions.
Backend	FastAPI + Pydantic	Validate requests and expose health, model-info and prediction endpoints.
Prediction service	pandas + joblib	Prepare input, load artefact, predict probability, apply threshold and business rules.
ML pipeline	scikit-learn Random Forest	Impute, engineer, transform/encode/scale and predict.

11.2 End-to-End Flow

- User enters known session fields in Streamlit.
- Frontend sends JSON to POST /predict.
- Pydantic validates values and categories.
- Missing fields become NaN and are handled by the saved pipeline.
- Random Forest generates purchase probability.
- Threshold 0.381 creates the binary decision.
- Service returns intent band, recommendation, imputation warning and key global factors.
- Frontend displays the result in business-friendly language.

12. Backend and Frontend Development

12.1 Backend

FastAPI exposes GET /health, GET /model-info and POST /predict. All 17 fields are optional, but at least five must be supplied. Numeric ranges and categorical values are validated. The service loads models/final_model.joblib, calls predict_proba and returns purchase_probability, will_purchase, decision_threshold, intent_level, recommended_action, fields_imputed, low_confidence, key_factors and model.

12.2 Frontend

The frontend was developed using **Streamlit** to provide a simple and interactive interface for entering online shopping session information and viewing the model's prediction. The design focuses on usability for non-technical users by organizing inputs into clear sections and presenting the prediction result in an easy-to-understand format.

The interface groups browsing behaviour, session metrics and visitor context. It provides casual-browser and serious-shopper presets, allows numerical fields to be marked **Unknown**, shows a probability gauge, and displays the predicted class, intent level, decision threshold and recommended action. If eight or more fields are imputed, the system displays a low-confidence warning to inform the user that the prediction is based on limited input information.

Deploy

Load an example session

Serious shopper

Tip: tick *Unknown* to leave a field out; the model imputes it.

Account/admin pages viewed

3

Unknown

Time on admin pages (s)

0.00

Unknown

Info pages viewed

1

Unknown

Time on info pages (s)

0.00

Unknown

Product pages viewed

45

Unknown

Time on product pages (s)

2100.00

Unknown

Bounce rate (0-1)

0.01

Unknown

Exit rate (0-1)

0.02

Unknown

Page value

35.00

Unknown

Closeness to special day (0-1)

0.00

Unknown

Month

Nov

Visitor type

Returning_Visitor

Weekend

Predict purchase intent

The figure shows the main **Streamlit user interface** developed for the Online Shopper Purchase-Intent Predictor. The form is divided into three clear sections: **Browsing Behaviour**, **Session Metrics**, and **Visitor Context**, making it easier for users to enter the available information about a shopping session. Fields that are not known can be marked as **Unknown**, allowing the preprocessing pipeline to handle the missing values automatically. The interface also provides an example-session option to quickly demonstrate how the system works. Once the required information is entered, the user can click the **“Predict purchase intent”** button to send the data to the backend and generate the final purchase-intent prediction.

Deploy

Load an example session

Serious shopper

Tip: tick *Unknown* to leave a field out; the model imputes it.

60% purchase probability

Prediction

Will buy ☒

Intent level

Medium-high intent

Decision threshold

38%

Recommended action: Probable buyer. Show social proof or free-shipping message to close the sale.

Key factors behind this prediction:

Based on the model's overall feature importance — which of the fields you gave it matter most in general.

- Page value: 35.0
- Exit rate: 0.02
- Month: Nov
- Time on product pages (s): 2100.0
- Product pages viewed: 45

Estimated automatically (not provided): Operating system, Browser, Region, Traffic type

Model: RandomForest

IT3051 – Fundamentals of Data Mining | Online Shoppers Purchase-Intent Prediction

The figure shows the prediction result generated after the user submits a shopping session. In this example, the model estimates a **60% purchase probability**, which is above the optimized **38% decision threshold**, so the session is classified as “**Will buy**” with a **Medium-high intent** level.

The interface also provides a business recommendation based on the predicted intent. For this case, it suggests using **social proof or a free-shipping message** to help complete the sale. To improve interpretability, the system displays the main factors influencing the prediction, including **Page Value, Exit Rate, Month, Time on Product Pages, and Product Pages Viewed**.

The interface also identifies fields that were not provided by the user and were automatically estimated by the preprocessing pipeline, improving transparency. Finally, the result page shows that the prediction was generated using the **Random Forest** model.

12.3 Training-Serving Consistency and Input Robustness

A key design decision was to serialize the preprocessing and estimator together. The API does not manually duplicate training transformations; it passes a single-row DataFrame into the same fitted pipeline used during modelling. This prevents training-serving skew caused by different encodings, scaling rules, feature engineering or imputers being used in production.

Missing optional fields are represented as NaN and handled by the fitted pipeline. Pydantic performs request-level validation before the model is called, while the service layer applies the optimized threshold and converts raw probability into an intent band, recommendation and confidence warning. This separates model inference from user-facing business logic.

12.4 Recommendation Rules

Intent	Rule	Action
High	$p \geq \max(0.60, \text{threshold})$	Do not over-discount; reduce friction and reassure checkout/stock/delivery.
Medium-high	$p \geq \text{threshold}$	Use social proof or free-shipping message.
Undecided	$p \geq 0.5 \times \text{threshold}$	Use targeted offer, recommendations or cart reminder.
Low	Below $0.5 \times \text{threshold}$	Use low-cost retargeting or newsletter sign-up.

13. System Testing and Results

13.1 Functional Test Cases

Test	Condition	Expected result
Health	GET /health with artefact present	200; status ok and model metadata.
Valid prediction	POST /predict with ≥ 5 valid fields	200; probability, decision, intent and action.
Too few fields	Fewer than 5 supplied	422 with explanatory message.
Range validation	Invalid numeric range	422 validation error.
Missing optional fields	Some fields omitted	Prediction succeeds; omitted fields listed as imputed.
Low information	≥ 8 fields imputed	low_confidence=true and frontend warning.
Model unavailable	Artefact missing	503 instead of silent failure.

13.2 Predictive Test Results

On the untouched hold-out test set, the optimized-threshold model achieved accuracy 0.897, balanced accuracy 0.825, precision 0.656, recall 0.720 and F1 0.687. Threshold-independent performance was ROC-AUC 0.938 and PR-AUC 0.772. The hold-out PR-AUC is above the tuned CV estimate of 0.753, so these results do not show an obvious overfitting problem.

13.3 Deployment Artefacts

- `final_model_trainonly.joblib`: evaluation model corresponding to hold-out results.
- `final_model.joblib`: deployment model refit using all available cleaned data.
- `best_params.json`: selected hyperparameters.
- `backend/api.py` and `backend/service.py`: API and prediction logic.
- `frontend/streamlit_app.py`: user-facing application.

13.4 Test Strategy and Acceptance Criteria

System testing covered both predictive behaviour and software behaviour. Predictive testing used the untouched hold-out partition. Functional testing checked service health, valid prediction requests, insufficient input, out-of-range values, omitted optional fields, low-information requests and model-unavailable failure. A test was considered acceptable when the system returned the documented HTTP status and response fields without bypassing validation or silently substituting invalid values.

13.5 Reproducibility Checks

The project uses a fixed random state for reproducible splitting and experiments, a single approved dataset file, a fixed run order for the three notebooks, and a saved best-parameter record/model artefact. The final reported metrics correspond to the train-only evaluation model, while the deployment artefact is refit on all available cleaned data after evaluation. Keeping those two artefacts conceptually separate avoids reporting training-on-test performance as if it were hold-out performance.

14. Limitations and Future Improvements

- PageValues dominates prediction and limits claims about early-session use; train a separate model without it for true real-time intervention.
- The public historical dataset may not match a specific retailer or current customer behaviour; revalidate on recent organization-specific data.
- The positive class is small; future work can add probability calibration and cost-sensitive threshold analysis.
- Displayed key factors are based on global importance, not per-prediction causal explanations; local SHAP explanations could be added carefully.
- Recommendation bands are rule-based and should be validated through controlled A/B testing.
- Expand automated unit/integration tests and continuous integration.
- Add post-deployment monitoring for drift, calibration, API reliability and model performance.

14.1 Additional Validity Considerations

Because the project missingness is synthetically generated under MCAR, conclusions about imputation performance should not automatically be generalized to Missing At Random (MAR) or Missing Not At Random (MNAR) production data. A real deployment should first characterize its own missingness mechanisms and retrain or redesign the pipeline if they differ.

- A future intervention model should optimize expected business value by combining conversion probability, customer value and intervention cost instead of relying only on F1 or PR-AUC.
- Probability calibration should be assessed before using predicted probabilities for financial decisions or campaign budgeting; strong ranking performance does not automatically imply perfectly calibrated probabilities.
- External validation on a second retailer or more recent e-commerce dataset would provide stronger evidence of generalization across time and organizations.

15. Group and Individual Contributions

This section records each member's work with reference to the project's Git branches and the specific files/sections they authored, reviewed, or ran.

15.1 Individual Contribution Statements

Member / Student ID	Contribution
Vihigum M. H. D. C / IT23699380	Defined the problem scenario and stakeholder requirements, prepared the dataset proposal, conducted the full exploratory data analysis (structure, missing-value overview, duplicates, class balance, distributions, categorical relationships), built and tested the Streamlit frontend (presets, probability gauge, key-factor display, low-confidence warning), and analysed and tuned the Logistic Regression and Decision Tree models.
Dewmin W. A. V / IT23733930	Investigated the four columns with synthetic missing values, confirmed the MCAR pattern via chi-square testing, designed and ran the controlled imputation-accuracy experiment (mean/median/KNN/iterative) and the downstream model-performance comparison, analysed numeric outliers and skew, and analysed and tuned the HistGradientBoosting and XGBoost models.
Vidyanjani H. M. R. M / IT23690202	Designed and evaluated the six engineered features, ran the feature-selection ablation (mutual information vs. Random Forest importance), led the PageValues leakage investigation and scope decision, analysed and tuned the SVM and KNN models, and produced the permutation-importance and SHAP interpretation of the final model.
H.N. Hapuarachchi / IT23713062	Designed and implemented the core preprocessing pipeline (src/preprocessing.py) and the shared model-development/tuning infrastructure used across both modelling notebooks, analysed and tuned the Random Forest model, selected and justified the final model, optimized the decision threshold, ran the hold-out evaluation, built the FastAPI backend and prediction service, and coordinated the repository structure, branch merges and end-to-end system integration.
All Members	All Members jointly reviewed the model-development process and optimization decisions, validated the frontend and backend implementation, participated in system testing, contributed to final report preparation, and carried out final quality assurance to ensure the overall solution was technically consistent, complete, and ready for submission.

16. Conclusion

This project completes the required data-mining workflow for online shopping purchase-intent prediction. Evidence-driven EDA and preprocessing addressed missing data, duplicates, class imbalance, skew and leakage risk. Seven algorithm families were compared and Random Forest was selected after tuning because it achieved the best cross-validated PR-AUC while remaining robust and interpretable.

The final model achieved ROC-AUC 0.938, PR-AUC 0.772 and F1 0.687 on the hold-out test set at the optimized threshold. It was integrated into a FastAPI backend and Streamlit frontend with validation, missing-input handling, confidence warnings and business-oriented outputs. The strongest limitation is the dependence on PageValues, which should guide the scope of deployment and future early-session model development.

References

- [1] Sakar, C. & Kastro, Y. (2018). Online Shoppers Purchasing Intention Dataset [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.24432/C5F88Q>
- [2] UCI Machine Learning Repository. Online Shoppers Purchasing Intention Dataset. <https://archive.ics.uci.edu/dataset/468/online+shoppers+purchasing+intention+dataset>
- [3] IT3051 – Fundamentals of Data Mining. Mini Project Assignment 2026.
- [4] Project repository artefacts: notebooks, experiment tables, figures, trained models, backend and frontend source code.
- [5] Sakar, C. O., Polat, S. O., Katircioglu, M., & Kastro, Y. (2019). Real-time prediction of online shoppers' purchasing intention using multilayer perceptron and LSTM recurrent neural networks. *Neural Computing and Applications*, 31, 6893-6908.
- [6] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.
- [7] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785-794.
- [8] FastAPI Documentation. <https://fastapi.tiangolo.com/>
- [9] Streamlit Documentation. <https://docs.streamlit.io/>